LINKED LISTS & TREES Solutions

COMPUTER SCIENCE MENTORS 61A

October 5, 2026 – October 9, 2026

1 Linked Lists

For the following questions, use this dataclass-based linked list. A `LinkedList[T]` is either a `Link[T]` or the empty tuple `()`. The type variable `T` describes the element type.

```
from __future__ import annotations
from dataclasses import dataclass

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self) -> str:
        return format_link(self)

def format_link[T](s: Link[T]) -> str:
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), 'Not a valid linked list'
    return string + ')'
```

Construct a list with `Link(1, Link(2))`. Read its elements with `first` and `rest`. At the end, `rest` is `()`, which has no `first` or `rest` attributes. **`print`** uses parentheses around the elements; entering a `Link` at the prompt shows the dataclass representation with field names.

Created by Cadence Loh, Ana Abell, Tien-Yu Hsu, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. Write `append`, which takes two linked lists and returns a new linked list with all of the elements of the first followed by all of the elements of the second. Then use `append` to write `reverse`, which returns a new linked list with the elements in reverse order.

```
def append[T](a: LinkedList[T], b: LinkedList[T]) -> LinkedList[T]:
    """
    >>> print(append(Link(1, Link(2)), Link(7, Link(8))))
    (1 2 7 8)
    >>> print(append((), Link(5)))
    (5)
    >>> append((), ())
    ()
    """
    if ____________________________________________:

        return ____________________________________

    return ____________________________________________________________


def reverse[T](lnk: LinkedList[T]) -> LinkedList[T]:
    """
    >>> print(reverse(Link(1, Link(2, Link(3)))))
    (3 2 1)
    >>> reverse(())
    ()
    """
    if ____________________________________________:

        return ____________________________________

    return ____________________________________________________________
```

```
def append[T](a: LinkedList[T], b: LinkedList[T]) -> LinkedList[T]:
    if not isinstance(a, Link):
        return b
    return Link(a.first, append(a.rest, b))

def reverse[T](lnk: LinkedList[T]) -> LinkedList[T]:
    if not isinstance(lnk, Link):
        return ()
    return append(reverse(lnk.rest), Link(lnk.first))
```

2. Write `keep_if`, which takes a linked list of integers `lnk` and a predicate `pred`, and returns a new linked list of only the elements for which `pred` returns true, in their original order.

```
from collections.abc import Callable
def keep_if(
    lnk: LinkedList[int], pred: Callable[[int], bool]
) -> LinkedList[int]:
    """
    >>> lnk = Link(1, Link(2, Link(3, Link(4))))
    >>> print(keep_if(lnk, lambda x: x % 2 == 0))
    (2 4)
    >>> print(keep_if(lnk, lambda x: x > 0))
    (1 2 3 4)
    >>> keep_if(lnk, lambda x: x > 10)
    ()
    """
    if ____________________________________________:

        return ____________________________________

    rest = ____________________________________________

    if ____________________________________________:

        return ____________________________________

    return ____________________________________________
```

```
def keep_if(
    lnk: LinkedList[int], pred: Callable[[int], bool]
) -> LinkedList[int]:
    if not isinstance(lnk, Link):
        return ()
    rest = keep_if(lnk.rest, pred)
    if pred(lnk.first):
        return Link(lnk.first, rest)
    return rest
```

3. Write `merge`, which takes two linked lists of integers, each sorted in increasing order, and returns a new sorted linked list containing all of the elements of both.

```
def merge(a: LinkedList[int], b: LinkedList[int]) -> LinkedList[int]:
    """
    >>> a = Link(1, Link(5))
    >>> b = Link(2, Link(3, Link(8)))
    >>> print(merge(a, b))
    (1 2 3 5 8)
    >>> print(merge(a, ()))
    (1 5)
    >>> merge((), ())
    ()
    """
    if ___________________________________________:

        return ___________________________________

    elif ___________________________________________:

        return ___________________________________

    elif ___________________________________________:

        return ___________________________________

    else:

        return ___________________________________
```

```
def merge(a: LinkedList[int], b: LinkedList[int]) -> LinkedList[int]:
    if not isinstance(a, Link):
        return b
    elif not isinstance(b, Link):
        return a
    elif a.first <= b.first:
        return Link(a.first, merge(a.rest, b))
    else:
        return Link(b.first, merge(a, b.rest))
```

2 Trees

For the following questions, use this dataclass-based tree. A `Tree[T]` has a `label` of type `T` and a list of `branches`, each of which is also a `Tree[T]`. A tree is never empty.

```
from __future__ import annotations
from dataclasses import dataclass, field

@dataclass
class Tree[T]:
    """A Tree has a label (of type T) and a list of branches,
    which are trees.

    >>> t = Tree(3, [Tree(1, []), Tree(2, [Tree(1, []), Tree(1, [])])])
    >>> t.branches[0]
    Tree(label=1, branches=[])
    >>> len(t.branches)
    2
    >>> t.branches[1].label
    2
    >>> len(t.branches[1].branches)
    2
    >>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'), Tree(3)]),
    ...                  Tree('+'), Tree(4)])
    >>> print(expr)
    10
      6
        2
        *
        3
      +
      4
    """
    label: T
    branches: list[Tree[T]] = field(default_factory=list)  # branches
       defaults to []

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

def is_leaf(t: Tree) -> bool:
    "Return whether a Tree t is a leaf with no branches."
    return not t.branches
```

4. What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue.

```
>>> t = Tree(5, [Tree(3, [Tree(8)]), Tree(1), Tree(7)])
>>> t.label

5

>>> t.branches[0].label

3

>>> is_leaf(t.branches[1])

True

>>> len(t.branches)

3

>>> t.branches[1].branches[0]

IndexError: list index out of range

>>> t.branches[0].branches

[Tree(label=8, branches=[])]

>>> u = Tree(2, [t.branches[0]])
>>> u.branches[0].branches[0].label

8

>>> print(u)

2
  3
    8
```

5. Implement `tree_height`, which takes in a tree and returns its height: the depth of the lowest leaf, where the root has depth 0.

```
def tree_height(t: Tree[int]) -> int:
    """
    >>> tree_height(Tree(1))
    0
    >>> t = Tree(1, [Tree(4), Tree(2, [Tree(3)])])
    >>> tree_height(t)
    2
    """
    if _____________________________________________:

        return _____________________________________

    return 1 + ____________________________________________________________________
```

```
def tree_height(t: Tree[int]) -> int:
    if is_leaf(t):
        return 0
    return 1 + max([tree_height(b) for b in t.branches])
```

6. Write `all_paths`, which takes a tree and returns a list of all root-to-leaf paths. Each path is a linked list of the labels from the root down to a leaf.

```
def all_paths[T](t: Tree[T]) -> list[LinkedList[T]]:
    """
    >>> t = Tree(1, [Tree(2), Tree(3, [Tree(4), Tree(5)])])
    >>> for p in all_paths(t):
    ...     print(p)
    (1 2)
    (1 3 4)
    (1 3 5)
    >>> all_paths(Tree(7))
    [Link(first=7, rest=())]
    """
    if _____________________________________________:

        return _____________________________________

    return ____________________________________________________________________
```

```
def all_paths[T](t: Tree[T]) -> list[LinkedList[T]]:
    if is_leaf(t):
        return [Link(t.label)]
    return [Link(t.label, p) for b in t.branches for p in all_paths(b)]
```

7. Write `count_paths`, which takes a tree `t` of integers and an integer `total`, and returns the number of root-to-leaf paths whose labels sum to `total`.

```
def count_paths(t: Tree[int], total: int) -> int:
    """
    >>> t = Tree(1, [Tree(3), Tree(3, [Tree(4), Tree(0)]), Tree(4,
        [Tree(1)])])
    >>> count_paths(t, 4)
    2
    >>> count_paths(t, 8)
    1
    >>> count_paths(t, 6)
    1
    >>> count_paths(t, 5)
    0
    """
    if ____________________________________________:

        return ____________________________________

    return ____________________________________________________________________
```

```
def count_paths(t: Tree[int], total: int) -> int:
    if is_leaf(t):
        return 1 if t.label == total else 0
    return sum([count_paths(b, total - t.label) for b in t.branches])
```